King Saud University – CCIS – CSC113

Phase 2 Project – Restaurant Management System

Instructor: Dr Mohammed Faroun

Team Members:

1) Student Name (ID):

    FARIS FALEH ALHARBI (445102358)

2) Student Name (ID):

    FAISAL SAUD ALSAHLI (445100823)

3) Student Name (ID):

    ABDULLAH OMAR ALMOKAITEEB (445100329)

Work Division:

1) FARIS: Worked on the GUI (RestaurantGUI and MenuFrame), the Restaurant class, and the file handling features, and UML Phase2.

2) FAISAL: Implemented the inheritance classes: Person, Employee, Waiter, and Customer.

3) ABDULLAH: Implemented the Dish and Order classes, the custom exception (InvalidDishException), and the Main class.

Introduction:

This report presents Phase 2 of the Restaurant Management System project.
In this phase, several new concepts were added to the system including exception handling, file operations, a Graphical User Interface (GUI), and replacing arrays with LinkedLists.
The goal of this phase is to enhance the design of the system, improve user interaction, and demonstrate advanced Object-Oriented Programming concepts.

Implementation Details:

The system was improved in Phase 2 by adding four main features:

GUI:

Two GUI frames were created using Swing.
RestaurantGUI handles input (add, save, load, show), and MenuFrame displays the menu.
Action events were used to connect the buttons to the backend.

LinkedList:

Arrays from Phase 1 were replaced with LinkedList to store dishes and orders dynamically.

File Handling:

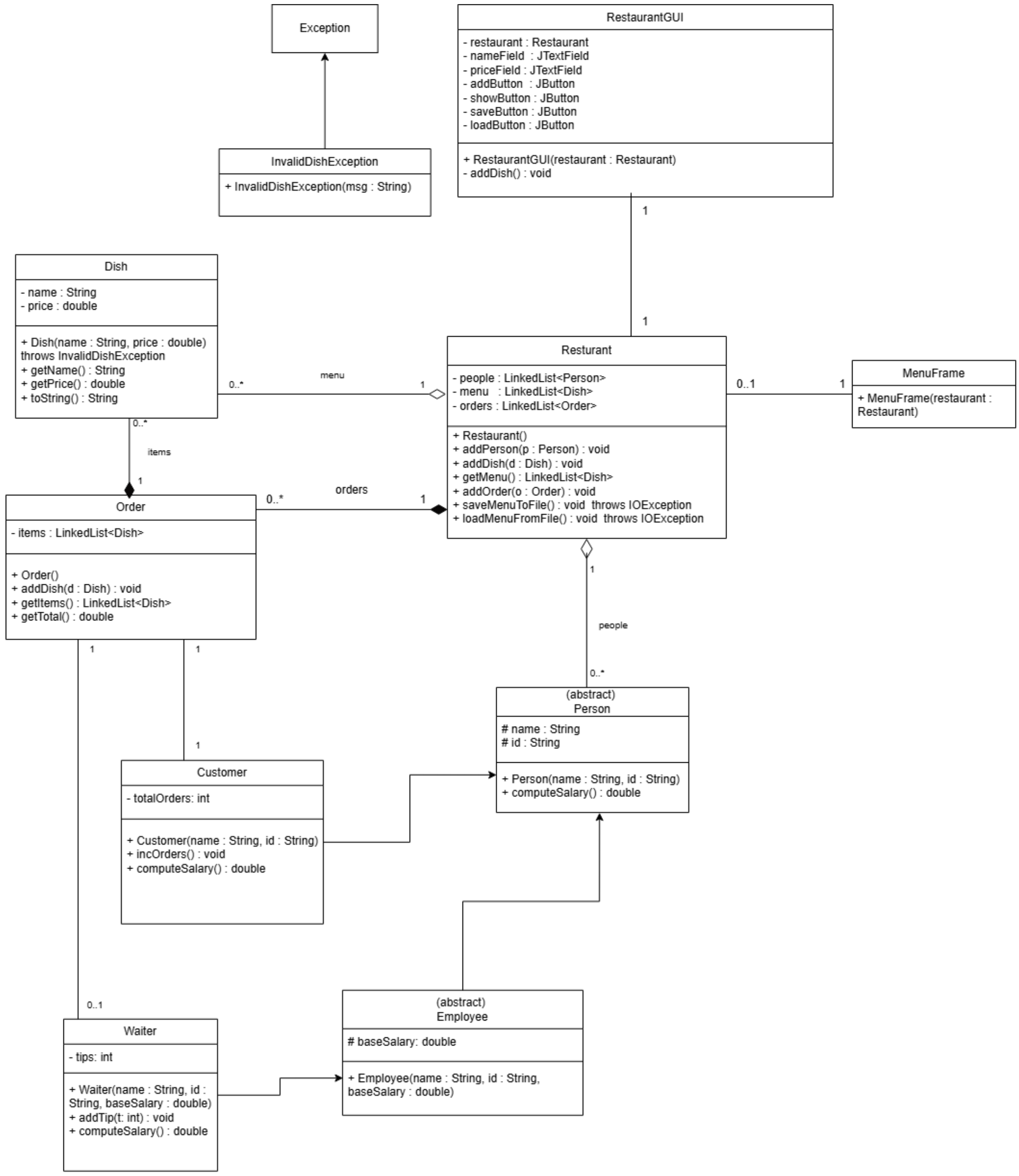
The Restaurant class includes saveMenuToFile() and loadMenuFromFile() to store and retrieve menu data using text files.

Exception Handling:

The system uses:

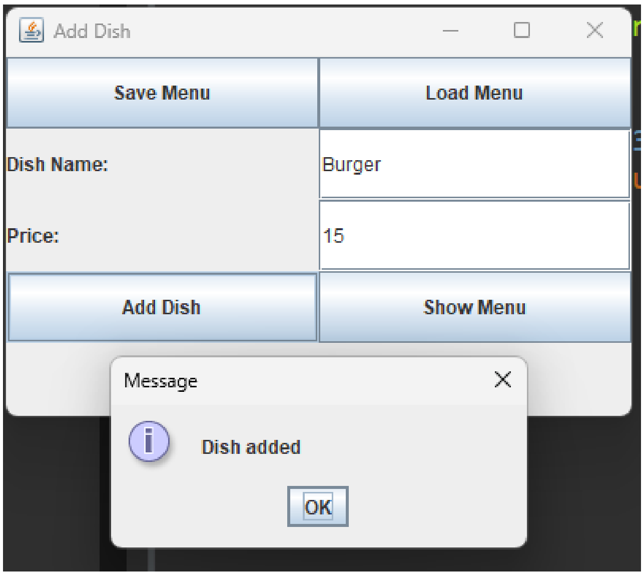
- **IOException** (checked) for file operations
- **NumberFormatException** (unchecked) for invalid numeric input
- **InvalidDishException** (user-defined) for negative prices

UML:


Exception
InvalidDishException
+ InvalidDishException(msg : String)
RestaurantGUI
- restaurant : Restaurant
- nameField : JTextField
- priceField : JTextField
- addButton : JButton
- showButton : JButton
- saveButton : JButton
- loadButton : JButton
+ RestaurantGUI(restaurant : Restaurant)
- addDish() : void
1
1
Dish
- name : String
- price : double
+ Dish(name : String, price : double) throws InvalidDishException
+ getName() : String
+ getPrice() : double
+ toString() : String
menu
0..*
1
Resturant
- people : LinkedList<Person>
- menu : LinkedList<Dish>
- orders : LinkedList<Order>
+ Restaurant()
+ addPerson(p : Person) : void
+ addDish(d : Dish) : void
+ getMenu() : LinkedList<Dish>
+ addOrder(o : Order) : void
+ saveMenuToFile() : void throws IOException
+ loadMenuFromFile() : void throws IOException
0..1
1
MenuFrame
+ MenuFrame(restaurant : Restaurant)
0..*
items
1
Order
- items : LinkedList<Dish>
+ Order()
+ addDish(d : Dish) : void
+ getItems() : LinkedList<Dish>
+ getTotal() : double
0..*
orders
1
1
people
0..*
(abstract)
Person
name : String
id : String
+ Person(name : String, id : String)
+ computeSalary() : double
1
1
1
Customer
- totalOrders: int
+ Customer(name : String, id : String)
+ incOrders() : void
+ computeSalary() : double
0..1
Waiter
- tips: int
+ Waiter(name : String, id : String, baseSalary : double)
+ addTip(t: int) : void
+ computeSalary() : double
(abstract)
Employee
baseSalary: double
+ Employee(name : String, id : String, baseSalary : double)

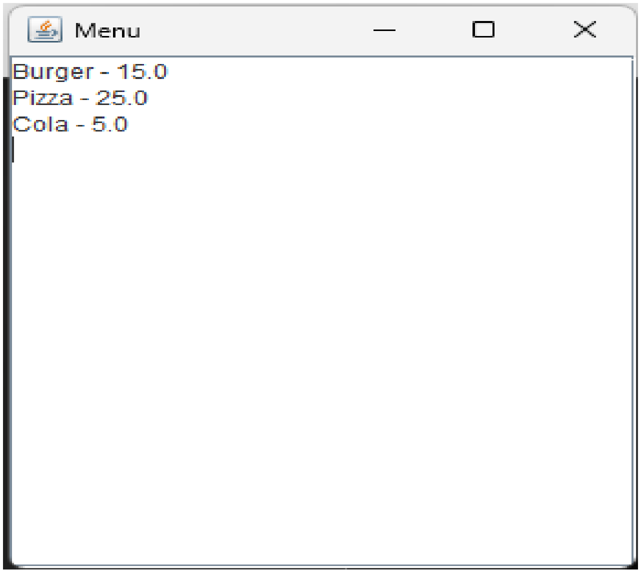
Screenshots:

1) Add Dish Window:



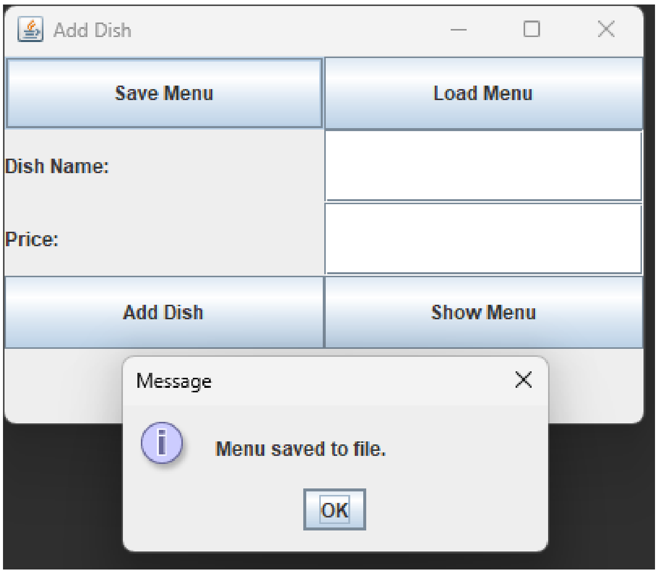
This screenshot shows the input frame where the user can add a new dish and access the menu, save, and load features.

2) Menu Display Window:



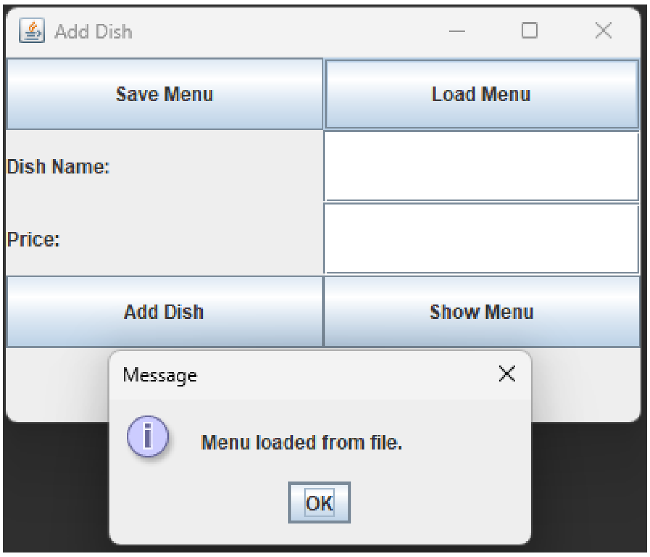
This screenshot shows the menu display frame, where all stored dishes are listed using a JTextArea.

3) Save Menu Confirmation:



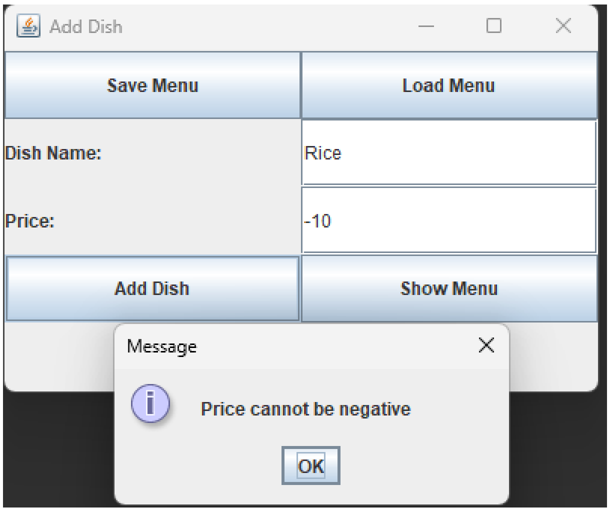
This screenshot shows the confirmation message displayed after successfully saving the menu to a text file.

4) Load Menu Confirmation:



This screenshot shows the confirmation message displayed after loading the menu data back from the file.

5) Exception Message (InvalidDishException):



This screenshot shows the error message triggered when entering a negative price, demonstrating the user-defined exception handling.

Conclusion:

In Phase 2, the Restaurant Management System was enhanced with several advanced features, including a graphical user interface, file handling, custom and built-in exception handling, and dynamic LinkedList structures.
These improvements made the system more flexible, user-friendly, and closer to real-world applications.
Overall, this phase demonstrates a solid understanding of key OOP concepts and successfully builds upon the foundation established in Phase 1.